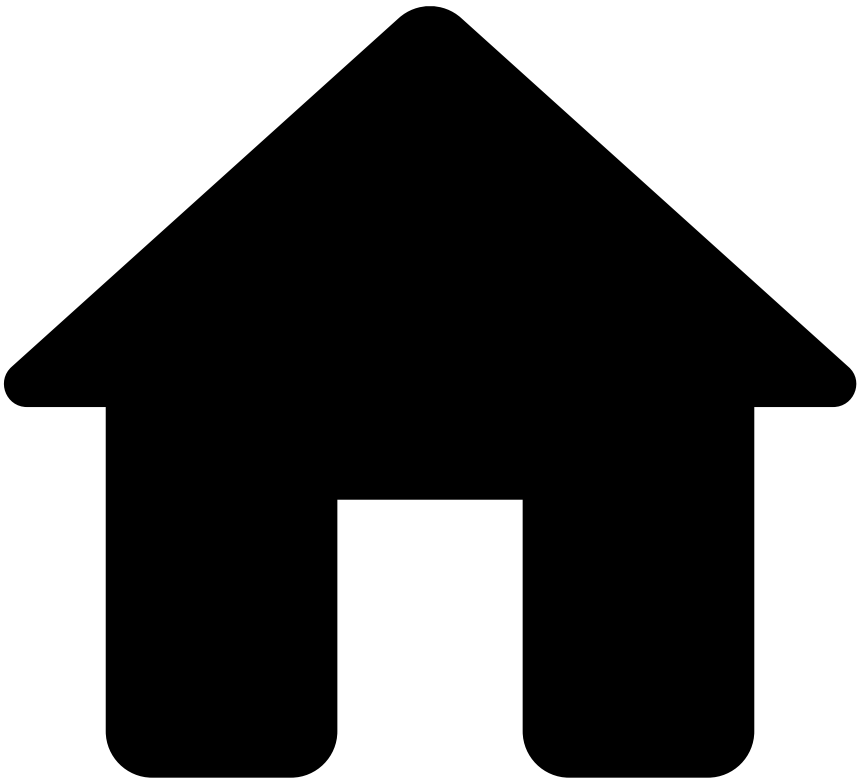


On this page

Workflows

Was this page helpful?Â 💬

"A workflow consists of a sequence of connected steps where each step follows without delay or gap and ends just before the subsequent step may begin." (*in Wikipedia*)

Workflows represent what Pulse executes in runtime. As Pulse integrates the data science concepts with their runtime counterparts, a workflow is the result of deploying those concepts directly into a Production environment. A workflow is where you can compose these concepts into a sequence of processing steps that produce a decision in real time.

As an example scenario, fraud detection is a task comprised of several steps with the aim of blocking fraudulent transactions. Usually, transactions must go through one or more sets of rules, one or more models for scoring and plans that perform transformations on data, in order to determine if they are fraudulent.

Thus, we group these processing steps into a workflow. Workflows represent the life cycle of an event through the system. They define the path that information traverses between components that may filter and/or transform the event.

If there is one thing essential to workflows, it is data. Without it, there is no sense in having workflows and all the remaining application resources. So, workflows always operate over an expected input data schema.

The following image displays a runtime workflow in Pulse:

Channelsâ□□

Each workflow typically corresponds to a particular channel through which the system receives information. For example, one workflow could be processing debit card transactions (debit channel), while another workflow would be dedicated to internet banking transfers (internet banking channel). Because different Channels typically have different Schemas (see below) and require a different set of rules and models, this approach ensures that each Workflow is optimized for the channel itâ□□s processing. For more information on channels, see Channels.

Outcomesâ□□

The ultimate goal of a runtime workflow is to produce a generic decision and insight on each event. Outcomes are flexible and use case-agnostic representations of those decisions. For example, *fraud/not fraud*, or *suspicious/not suspicious*.

A workflow can be configured with several outcomes for finer-grained decisions. For example, adding the type of fraud to events, instead of just classifying them as *fraud/not fraud*.

Pulse adds each outcome decision field (e.g *fraud*) and outcome score field (e.g. *fraud_score*) to the schema of events moving along the workflow.

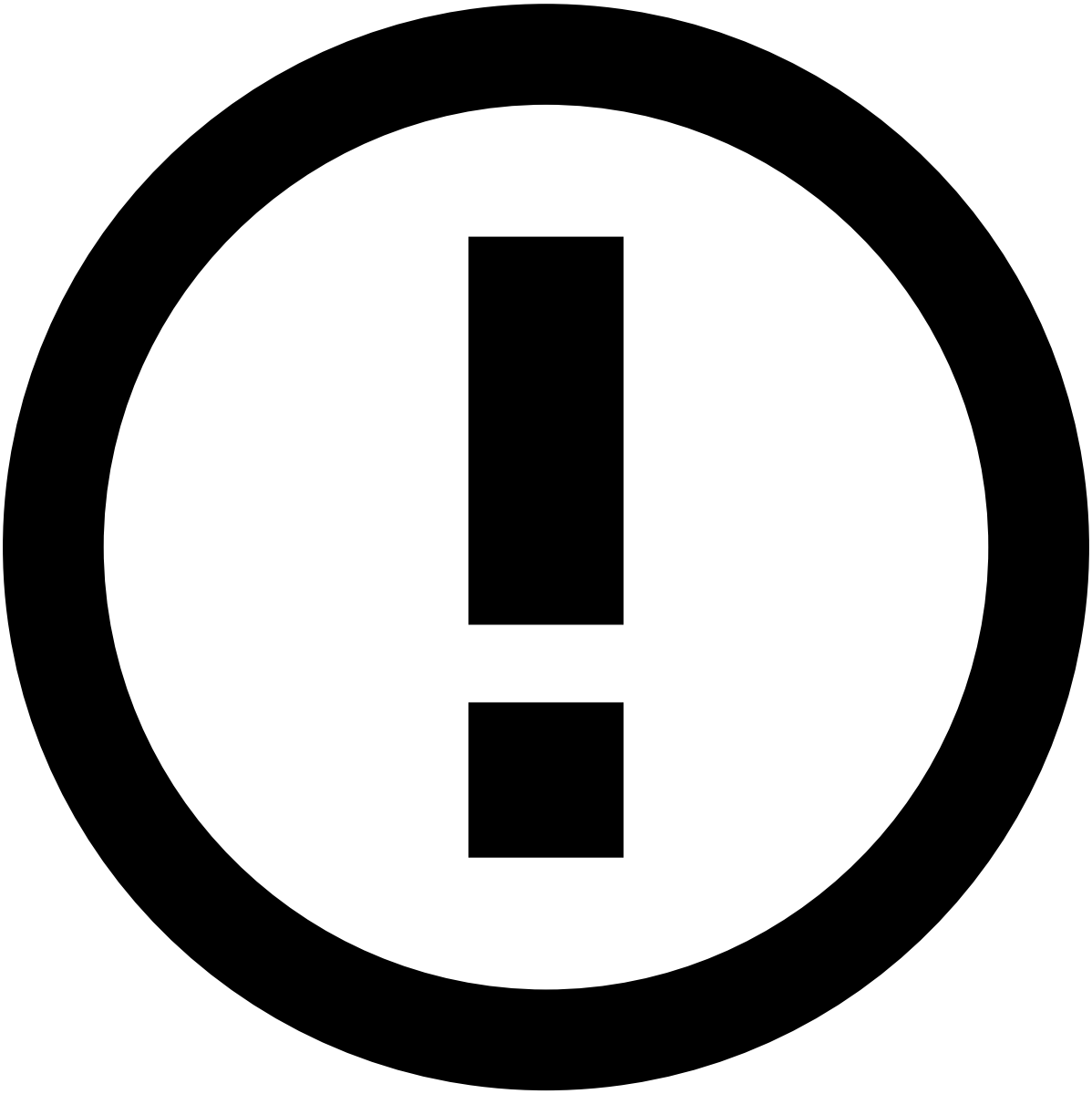
As you add elements such as scoring services and rules services, you must also configure how they affect these fields, i.e. how they contribute to each of the final decisions and scores of the workflow.

Another very important capability of outcomes is enabling you to manage feedback loops better, and to integrate with human validation. Whenever an event traverses a workflow, Pulse saves it in the event storage. Thus, it is possible to recover the Outcome of events at a later time to:

- Generate reports on model accuracy and outcome distribution for a consistent evaluation of runtime performance.
- Retrain models using the event storage as a source of data after receiving human input. This is especially important to close the loop on human feedback, for example, when Pulse classifies a transaction as fraud but a human analyst overrides the decision based on real-world evidence.

Workflow schemasâ□□

Runtime workflows in Pulse must always be configured with an origin schema. This schema, configured at the workflow level, defines the structure and formatting of the events that Pulse consumes from an external source of data.

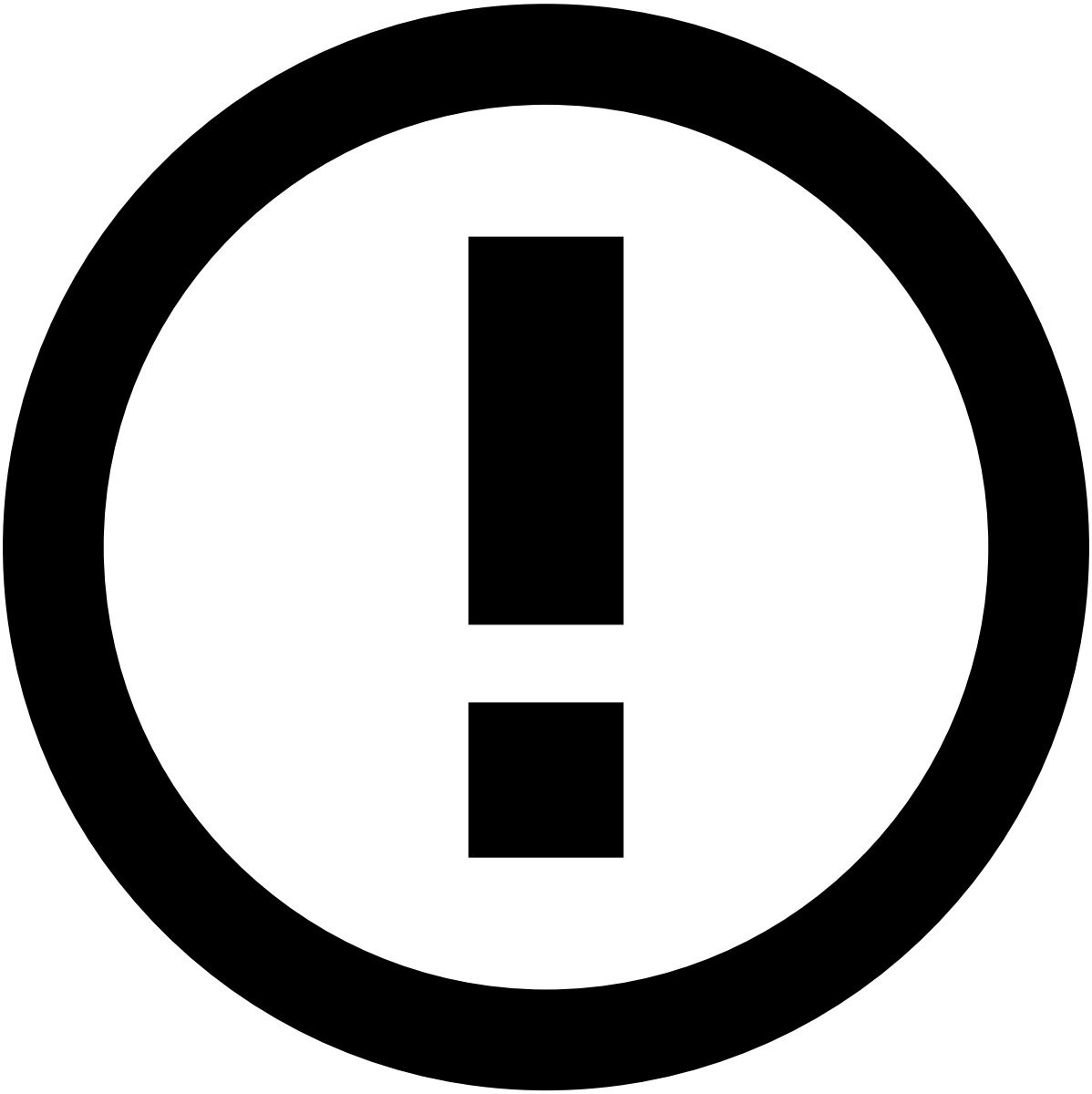


Workflow schema immutability

To ensure that the workflow is always capable of correctly consuming data, the origin schema is immutable.

However, as events flow through the different stages of the workflow, their schema can change. For example, new fields are added to the event when it is processed in a plan and enriched with features and profiles.

Additionally, if a workflow element only requires a subset of fields in the schema as input, only those are affected in the event once it receives the output of the element. The rest of schema fields remain unchanged and still available for subsequent workflow elements.



Mutable element schemas

Despite the fact that the schema you configure for input data in the workflow is immutable, the input and output schemas for each workflow element can be different.

This depends on the logic you implement for each workflow element, for example in a plan, by removing irrelevant fields, changing the names of fields to match project specifications, adding profiles, generating features and adding enrichment fields.

Conflictsâ□□

Since workflow schemas are mutable, when an event reaches a Workflow Service its schema may have been transformed previously by another service.

When configuring a workflow service, if Pulse is unable to match fields from the workflow schema to the input or output schemas of the service, it will generate conflicts and invalidate the workflow configuration. Pulse generates a conflict for

each field required by the service that is not present in the workflow schema with the same name and type. This can be due to:

- Input fields with different names, for example *merchant_name* and *mcn*.
- When you remove fields with a plan.
- When the service outputs a new field that already exists in the workflow schema.

In these cases, to fix your workflow, you must solve conflicts by manually mapping fields from the workflow schema to the input and output schemas expected in the service.

Removed fields

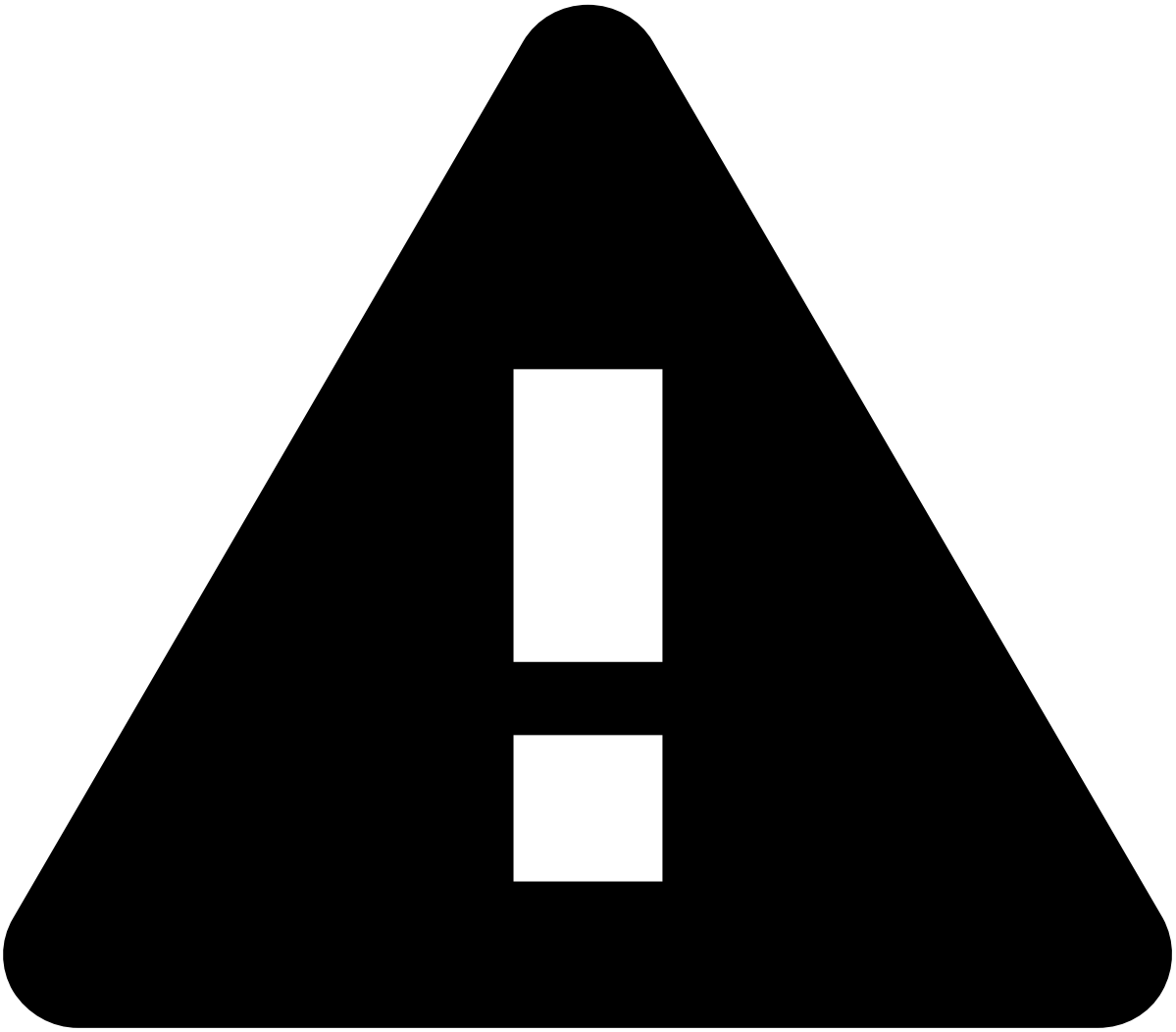
If a plan you configured for a plan service removes fields from the workflow schema, then you must ensure that subsequent services in the workflow do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your workflow, and you will have no available fields to map from.

In these cases, you should redesign your plan, articulate it with the remaining data science work to be used in runtime, and redeploy it to your workflow.

Extra fieldsâ□□

In runtime tasks, for example, when you design a workflow, Pulse automatically adds the following fields to the event schema:

- **timestamp**: The timestamp field is used to signal the date of a transaction in the UNIX Timestamp Format. See Time, schedules and timezones for more information.



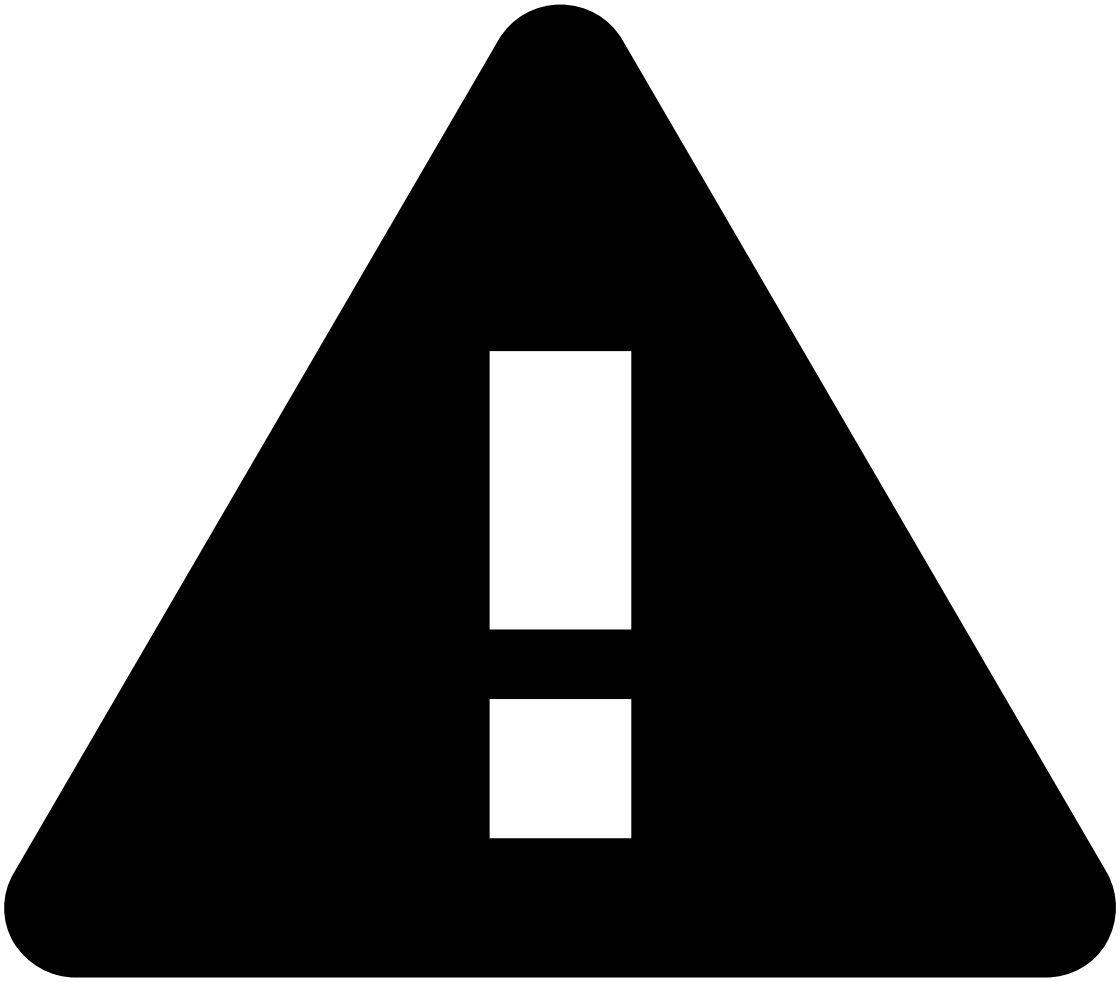
Timestamp

The value for this field must always be greater than or equal to the previous value processed since it's not possible to go back in time.

Reserved words

The word *timestamp* is reserved. Pulse does not allow you to use a metrics plan to calculate metrics over a field named *timestamp*.

- **UUID**: The unique id of the event. This field is used by several internal services and can also be used by external ones to uniquely identify an event, correlate results, access stored events, among others.



UUID

The system does not fill in this field automatically. The user building the application is responsible for setting it correctly when creating the events.

Reserved words

The word *uuid* is reserved. Pulse does not allow you to name any field name as *uuid*.

- **Outcome**: Outcomes represent decisions of your workflow, for example, *fraud/not fraud*. It is a categorical field used to mark an event as, for example, either fraudulent or not.
- **Outcome score**: The transaction score that corresponds to the workflow decision represented by the outcome. Scoring services and rules services contribute to outcome scores.

Workflow elementsâ□□

A workflow consists of processing steps (elements) and links between them. Each workflow element can process or transform the data as soon as the data arrives at it. Workflow elements can be seen as components that process data with the ability to apply operations such as filter and store.

Feedzai Pulse provides three main services:

- **Rules service**: Is a Pulse service used to declare multiple rules to detect and mark patterns, optionally contributing to an outcome.
- **Scoring service**: Is a module that feeds events into a model and produces a score, optionally contributing to an outcome.
- **Plan service**: Enables you to deploy data transformations you have defined and tested in plans at the data science stage, for example, to generate features before passing events onto the scoring service.

In addition to these core components provided off-the-shelf, Pulse also lets you include custom services in your workflows.

Rules serviceâ□□

The **rules service** allows defining the rules to be applied to a transaction event and the actions to be taken when the rule is matched. Additionally, a rules service can output a score and contribute to an outcome.

There are different mechanisms that can be applied when a given rule is matched:

- The corresponding event contributes to an outcome, similarly to how the record is immediately marked as a target value during the data science stage.
- Jump to the next workflow step, or even skip the remaining steps if the control flow of the rule is configured as *Stop Workflow*.
- A specific action can be triggered and processed by custom services. They can then apply specific modifications to the corresponding transaction event when handling the action.

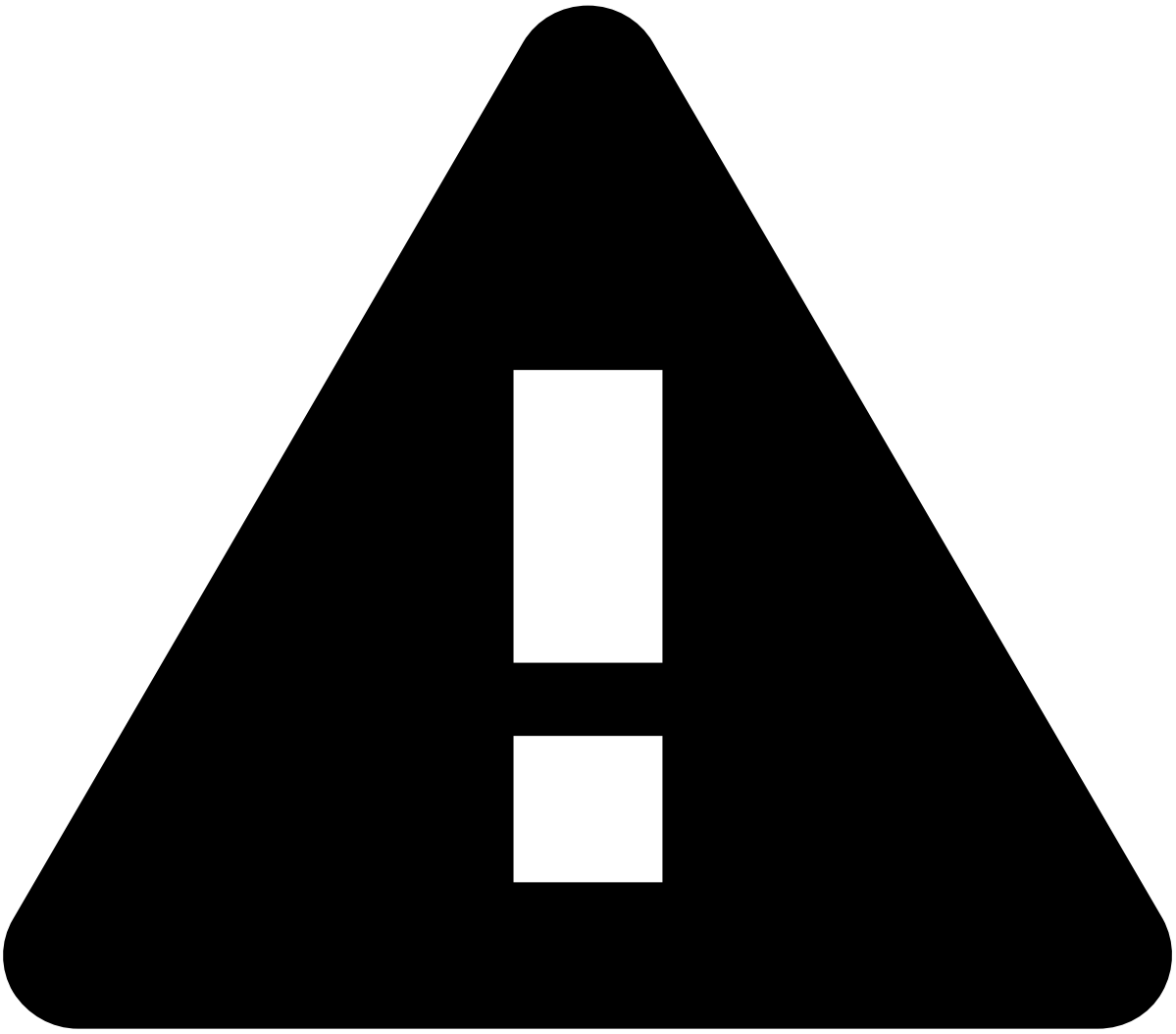
danger

The **rules service does not evaluate the rules** for every record that passes on Pulse. If the message respective to the record appeared due to a replica or recovery process and the rules project configuration is set to be stateless, then the rules will not be evaluated. This change is motivated by performance, since the calls to rules engine are expensive, avoiding some of the calls will reduce the execution time.

Scoring serviceâ□□

A scoring service executes a model that was previously trained during the data science loop. The job of a scoring service consists in **producing a score for each event** that goes through the service. For example, a score that denotes the likelihood of the event representing a fraudulent transaction.

Additionally, and depending on how you configure the scoring service, it will also **contribute to one of the outcomes** in the workflow. In some cases, it may make sense for you to have a deployed model without influencing the final decisions of the workflow. For example, you may want to temporarily deploy a model just to test its live performance for a few days/ weeks before phasing it in and start contributing to outcomes. In these cases, you can retrieve the scores, for example with a custom service, from the event storage or using Pulse's API.



White-box explanations in the scoring service

You may need to edit the scoring services that are in production since a previous Pulse version and update explanations with the latest version of white-box explanations (WBE).

Note that you can still use the older version of WBE, but you will not be able to change the critical path.

Due to the complexity of this type of workflow element, you must first understand models in Pulse in order to learn how scoring works.

Plan serviceâ□□

The plan service reflects, in production, all the work done at a data science stage.

A unique feature to Pulse is precisely enabling you to deploy feature plans into runtime without effort. This tight integration of data science and engineering represents a huge boost in performance and productivity.

Thus, the plan service takes a set of data transformations, for example, to generate features out of raw data, that have been thoroughly tested as part of the data science stage in your project and marked as fit to be used on live data.

Custom servicesâ□□

Custom workflow services are tailor-made programs that allow you to extend or modify Pulse's behavior to fit specific needs.

Their tasks can vary greatly in intent, but one common usage is to configure them as an input adapter. These services are an essential tool to feed data to a workflow. They can be used to build tailor-made connectors that integrate your custom data sources (for example: CSV files, databases, sockets, etc.) with the workflow inside Feedzai Pulse.

Another usage is, generically, when you might also want to modify the events as they make their way through the workflow to match some specific condition.

Designing a workflowâ□□

Workflows in Pulse offer a lot of flexibility in how you can combine elements and design a complex event processing pipeline in runtime.

However, most of the concepts you deploy to runtime in a workflow are originated during an offline data science stage. Thus, Pulse has to ensure the consistency of these concepts in a Pulse project, for example, deployable plans, and guarantee that they produce the same results both during the data science loop and during runtime.

Parallel forks in workflows

Pulse does not support parallel forks in your workflows. Since Pulse is unable to enforce the consistency at design-time, it will always force a real-time event to follow a single linear path from input until the workflow terminates.

If you configure branches as mutually exclusive, you are explicitly guaranteeing that an event can only go through one path. If the branches are parallel, then Pulse will automatically route the event through the first branch that evaluates as *true* and log a warning.

When joining branches into a single path at a workflow element, you must make sure that all incoming paths carry events with the exact same schema.

For more details on workflow design patterns and recommendations on how you can process events in runtime, check the User Guide.

Exampleâ□□

Let's look at a simple use case and see what can happen to events while they're flowing through a workflow. The following diagram shows how different transactions are processed by the workflow:

Designâ□□

Events have only two fields:

- amount
- country

The workflow is composed of the following elements:

- One input adapter. It connects to both rules services by forking the events flow through a filter: local and non-local transactions.
- Two rules services, for both local and foreign transactions.
- One scoring service.

Consider that for each rules service you are using rules based on the amount field.

- For the local transactions, the rules service executes a rule where the amount needs to be greater than 200
- For foreign transactions, the second rules service executes a rule where the amount is greater than 1000.

For both cases, the rules are configured to stop the workflow.

Executionâ□□

In this scenario, the events flow as follows:

1. The input adapter consumes the events.
2. The workflow forks the events whether they are local or not.
3. The corresponding rules service runs the rule block for each type of events.
4. Due to the **Stop Workflow** configuration, events that satisfy a rule condition will not be passed to other workflow elements.
5. Only two events reach the end of the workflow.

Workflow lifecycleâ□□

A workflow runs as a long-lived loop of reading and submitting events while managing the lifecycle of each workflow service as well. The workflow lifecycle is composed of three main moments as depicted in the following diagram:

- **onCreate**: While you add elements to the workflow, configure and connect them, Pulse instantiates all services and performs internal actions such as validating the input and output schemas. This stage is executed at workflow design time. While the workflow execution is not started, all services should be ready to receive recovery events.
- **onStart**: When you publish your Pulse application, the actual runtime begins, launching the workflow and starting to consume events through the input adapter. Additionally, Pulse assumes that all workflow and services configuration is valid. From this stage on, the workflow can only receive normal or replica events. For each service, depending on its type, Pulse executes specific logic to process events:
  - **Input adapter:** Run a single specific processing thread for the input adapter that consumes and submits events continuously while the workflow is running.
  - **Internal services:** Deliver messages to each service as a normal event traverses the workflow. After a service finishes processing the event it will reply indicating what the workflow should do with the event. For example, moving the event forward in the workflow and delivering a new message to the next workflow service, or dropping the event from the workflow and ending it.
- **onStop**: When the Pulse application stops, the workflow also stops receiving and processing events.

Destroying a workflow

The full lifecycle of a runtime workflow does not work in a cycle. This means that when you stop an application and its runtime processing, Pulse will not reset the state of the workflow to the previous *onCreate* stage.

In reality, when you stop an application, Pulse tears down the current internal representation of the workflow and creates a completely new one.

The workflow teardown includes the following tasks, in this order:

1. Stop the partitioning router from receiving messages

2. Stop the workflow synchronizer from receiving messages
3. Stop the workflow services and deregister jobs
4. Destroy workflow services
5. Close the partitioning router
6. Close the workflow connectors
7. Close the workflow synchronizer
8. Stop the workflow job manager
9. Close the executors
10. Close the connection to the event storage
11. Stop collecting performance metrics about the workflow

Workflow timeâ□□

Many workflow elements are tied to the notion of time, for example:

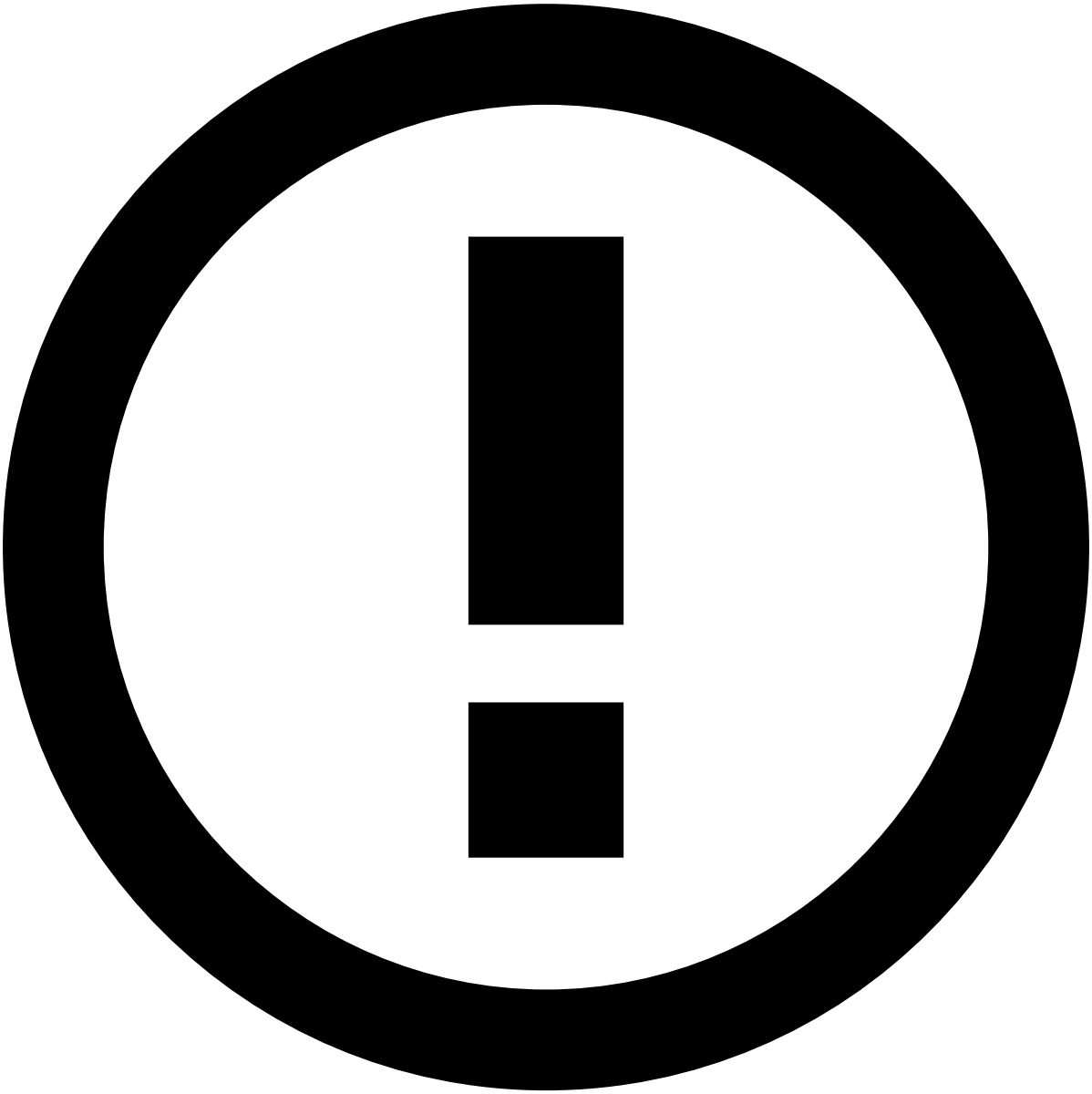
- Enabling a list item at a given time.
- Triggering a Scheduled Metrics Job at a given time.

The way Pulse workflows deal with time might not be intuitive at first, so itâ□□s crucial to understand the concepts of workflow clock and internal timezone. See Time, schedules and timezones for more information.

Message processingâ□□

Workflow recoveryâ□□

The recovery procedures for a workflow consists of replaying events which have already been processed and are present in the event storage. This is of the utmost importance since the underlying application may crash or fail for some reason, and upon its restart, all of its state is gone. For instance, you may have rules depending on windows, which will be empty and, thus, may be incorrectly evaluated.

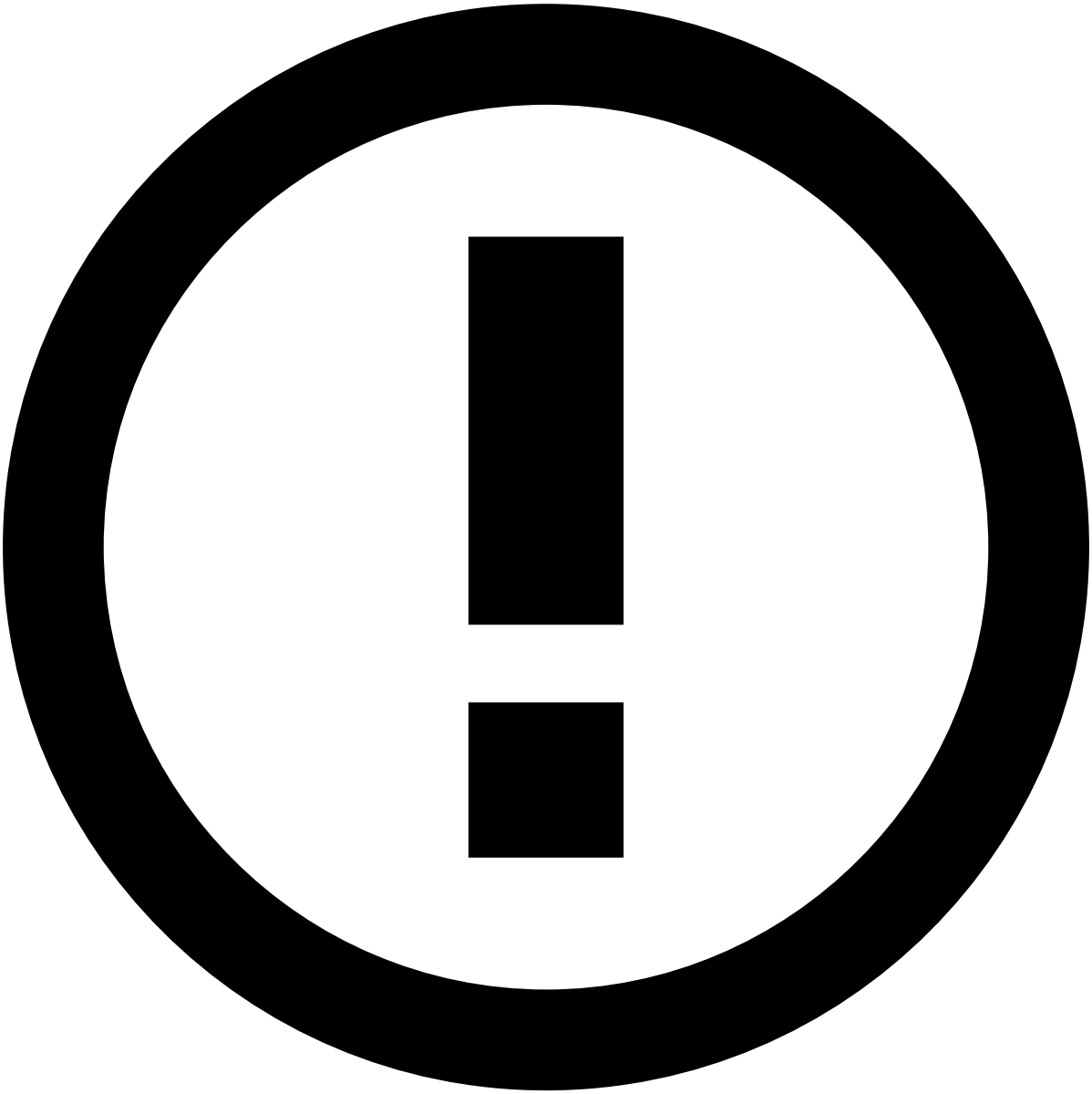


Event Storage

Note that these events are obtained from the event storage, so the event storage must be enabled in order for the recovery to work.

Furthermore, in a high availability environment you may have more than one replica of a given application, and if one fails Pulse must restore its state in order to have a consistent event evaluation.

An application may have several workflows, and when starting/restarting the application all the workflows that have the recovery option enabled will be recovered synchronously. This means that Pulse will publish the events to the respective workflows on a timestamp basis so that scheduled jobs will run correctly.



Specific considerations

There are some caveats to the recovery procedures:

- The event storage option must be enabled, otherwise, there is no event source;
- If a workflow event storage database table is empty or simply does not exist, that workflow will be ignored;
- In order for the recovery procedures to publish to the workflow, the workflow must contain an instance of a WorkflowInputService;
- The Fraud Models are not updated in recovery;
- The recovered events will not be stored in the event storage again.

Check how you can configure recovery for your workflows in the **Workflow recovery configuration** section of this topic.

Checkpoint-based workflow recoveryâ□

In a production environment, Pulse can be configured to store the necessary events for real-time metrics calculation on disk. In this case, Pulse creates **checkpoints**, i. e. regular snapshots about the events and the state of the metrics. After a server failure, Pulse can reload the last checkpoint and recover the metrics status starting from there. This saves time

when the server needs to be restarted. However, it takes toll on the metrics calculation speed, as Pulse puts metrics calculation on hold while creating a checkpoint. Hence, this setting needs to be carefully tested.

Limitations

Note that this feature is in continuous development, and as of now it does not provide full feature parity to the previous, RAM-based version.

Additionally, if this feature is enabled, do not edit or remove existing metrics. Edited metrics are not recoverable after a server failure, meaning that those metrics will only take new events into consideration after any server restart.

Workflow synchronizationâ□□

When in a distributed environment with more than one replica of an application, Pulse uses the underlying messaging platform to synchronize every message that reaches the workflow's dead letter to the other replicas.

Since it only sends the message in the dead letter, it has already been processed by the necessary workflow services, thus diminishing the work needed to be done in the other replicas, for instance not having to perform any scoring at a scoring service.

This behavior is similar to the workflow recovery procedures.

Message processing summaryâ□□

	Normal	Workflow Recovery	Workflow Synchronization
Actions	Yes		
Dead Letter	Yes	Yes	Yes
Explanations	Yes		
Event Storage	Yes		
Profile Update Jobs	Yes (only if leader)		Yes (only if leader)
Rule Evaluation	Yes	Yes	Yes
Scoring	Yes		

Interaction between workflowsâ□□

In order to enable Channels, you might want to share certain information between workflows. For example, in the workflow processing ATM withdrawals, you might want to consult the number of recent devices used by the same customer in internet banking (assuming ATM and internet banking events are processed in different workflows). This can be achieved with Omnichannel Plans. See Plans in workflows - Omnichannel metrics for more information on how these Plans work in the Workflow.

Learn moreâ□□

Check the following pages in the User's Guide to learn how to put workflows into practice:

- Configuring outcomes
- Adding input adapters
- Adding plans
- Adding models
- Adding rules
- Adding custom services
- Advanced workflow configurations
- Workflow services properties

Additionally, check:

- The **User's Guide** for more information on how Pulse projects work and how Data Science and runtime are integrated.